

敏感信息脱敏通用工具

纯 JAVA starter 组件
再见手艺人

AI 编程的理解

AI 编程的价值，不只是在生成代码，而是一整个闭环的交付流程。



工程交付

从问题定义到最终交付。



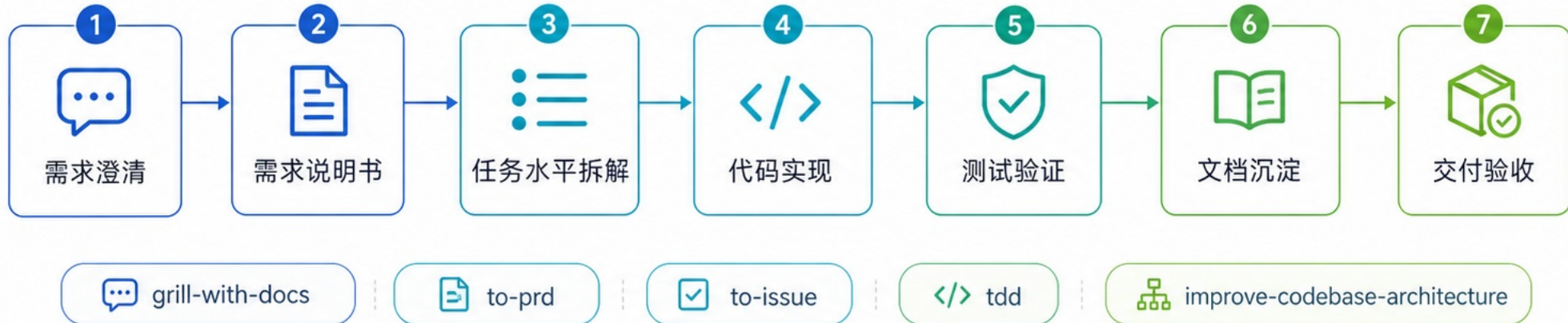
方法复用

用稳定工作流替代反复对话。



边界对齐

稳定的代码输出拉齐技术短板。



基于 108k stars 的 [mattpocock/skills](#) 理论扩展实验，沉淀出一套可复制的纯AI工程 workflow

脱敏组件痛点分析

输出脱敏是老系统长期重复出现的治理问题，通用组件研发具备挑战。

01



官方痛点



自行脱敏



写法不一致



显示与打印都要覆盖

02



调研阻力



老系统不一致



清单不完整



变量命名不统一

03



最终决策



组件必要



痛点可产品化



AI 降低试错成本



注解实现



配置实现



响应劫持



日志捕获



满足规范



AI助力

本项目希望验证：组件化工作能否借助纯AI实现，并最终交付。

如何提升纯 AI 项目的交付确定性

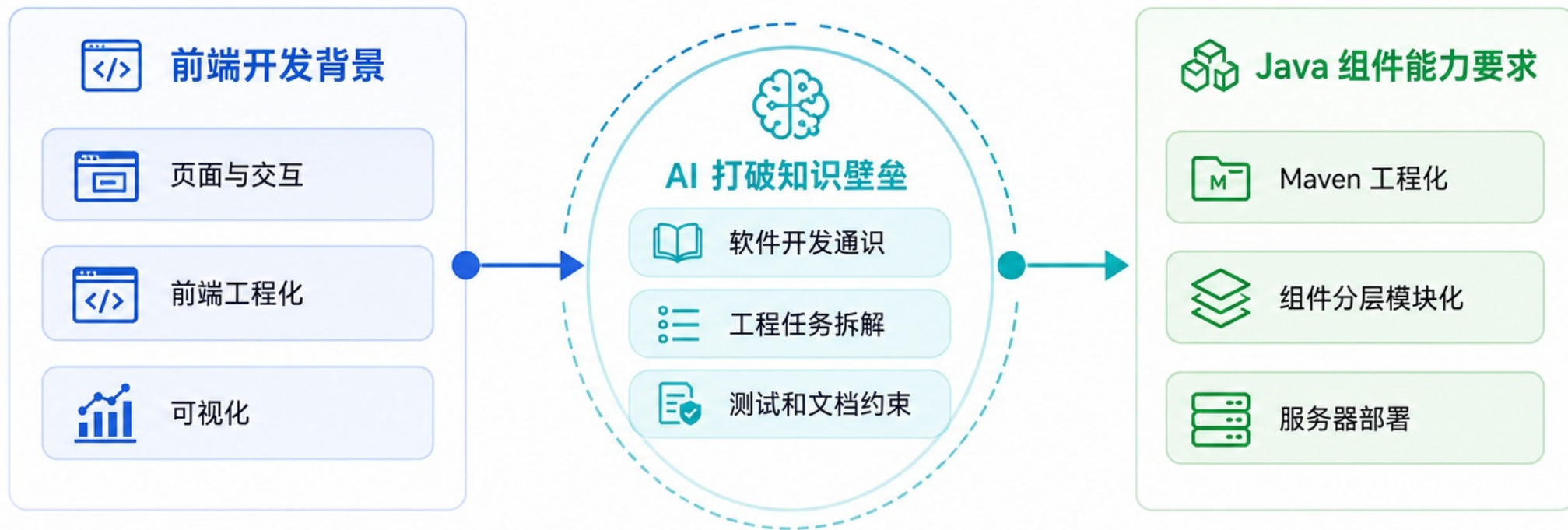
AI写一段代码我们都见识过，但如何让其稳定输出，再推进到闭环交付，是我们重点思考的。



边界可控、过程可记录、结果可验收，是本项目的交付前提。

队伍挑战：跨技术栈跨语言

队伍由前端开发组成，在 Java 战场上做 coding。



亮点：挑战组件全生命周期，前端演示 -> 后端开发 -> 服务器部署。

参赛作品：Safe Output

面向老系统的敏感信息脱敏通用工具

分散治理问题



各系统各写一套



规则口径不一致



清单不完整，遗漏难发现



Safe Output

Safe Output 是面向 Java 8 / Spring Boot 2.x 老系统的纯 Java starter，在少改业务代码的前提下统一处理输出侧脱敏。

三项交付特征



快速接入：

starter 自动装配，注解和配置降低改造成本。



主动覆盖：

覆盖常见敏感字段、接口返回和日志打印。



强化治理：

通过日志线索生成规则建议，进入持续回补。



从“分散治理”变成“统一组件 + 统一规则 + 可复查结果”。

覆盖、发现与回补闭环

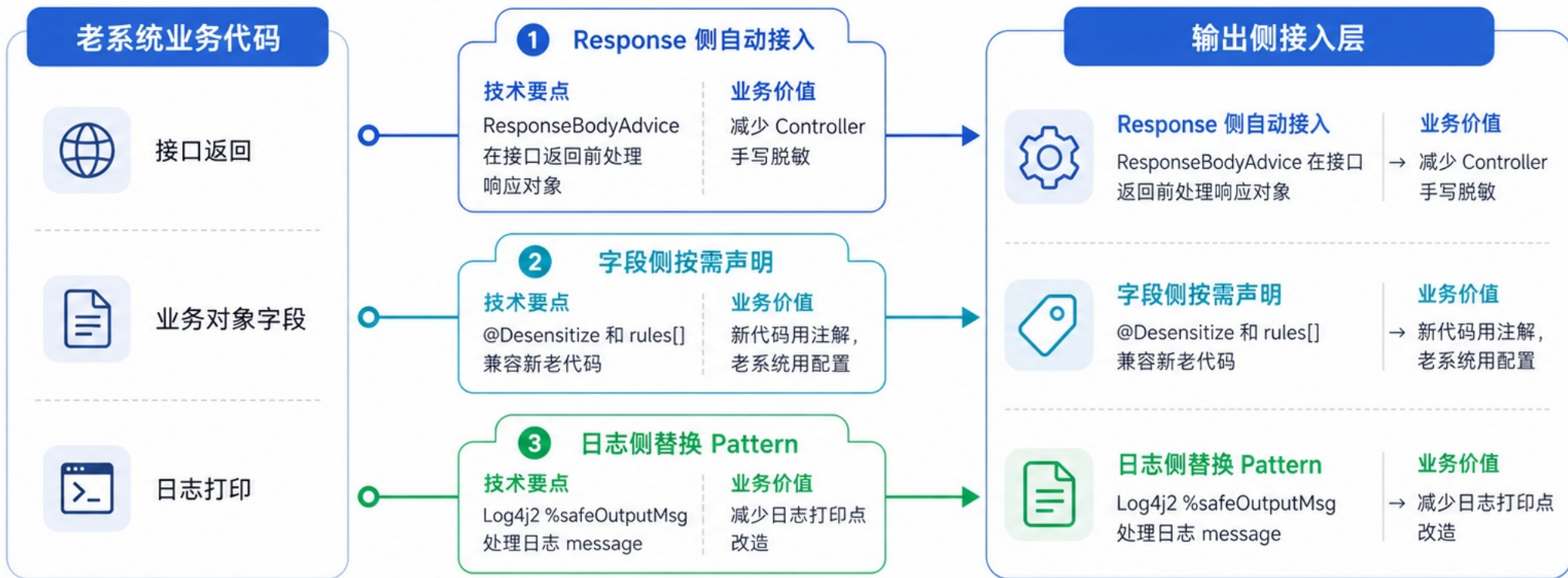
输出侧治理不宜只依赖一次性规则配置。



全流程只沉淀聚合统计和脱敏 evidence, 不保存敏感原文。

输出侧低侵入接入

快速接入的关键，是把改造点收敛到输出链路和规则配置上。



低侵入不是零改造，而是把改造面控制在 **starter、规则和日志 pattern** 上。

高频风险面覆盖

老系统难以一次拿到全量脱敏清单，先覆盖高频风险面更现实。



主动覆盖的目标是先纳入高频风险，再通过日志线索和规则建议持续补齐。

组件化架构设计

Safe Output 将规则、接入、日志、治理和展示能力拆分到不同模块。



老系统只需要接入 starter, 核心、日志和治理能力保持边界。

统一规则链路



统一规则链路让不同系统、不同输出场景按同一套口径治理。

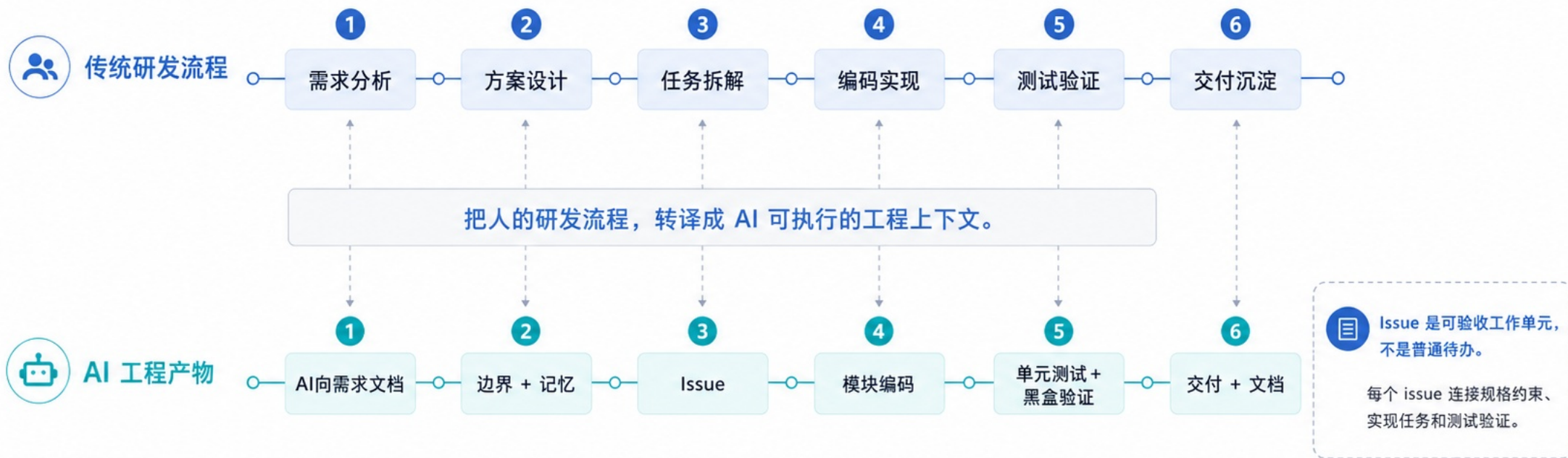
接入边界与运行护栏

组件只接管输出侧治理，不改业务主链路；默认配置限制扫描范围，异常时优先保证业务可用。



默认有界扫描 • 超长日志 fail-open • 兜底正则默认关闭

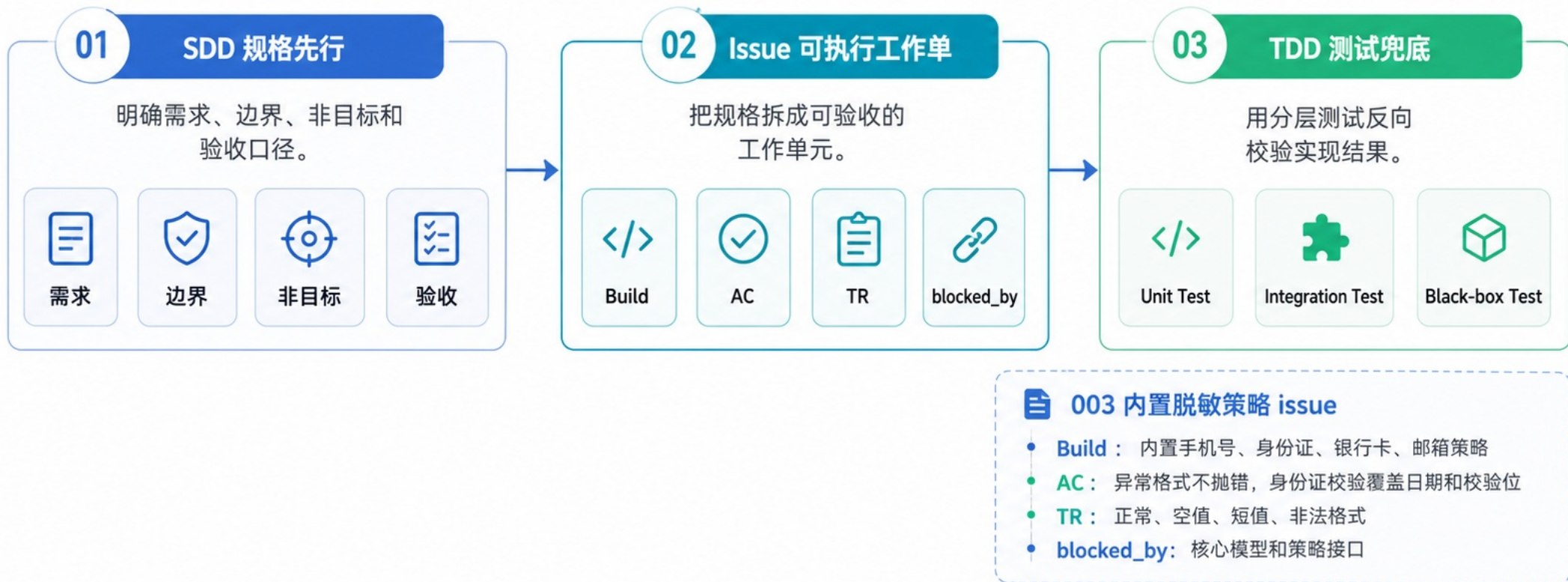
纯AI vs 传统研发



AI 不替代研发流程，而是加速执行其中可结构化、可验证、可沉淀的部分。

规格、任务与测试约束

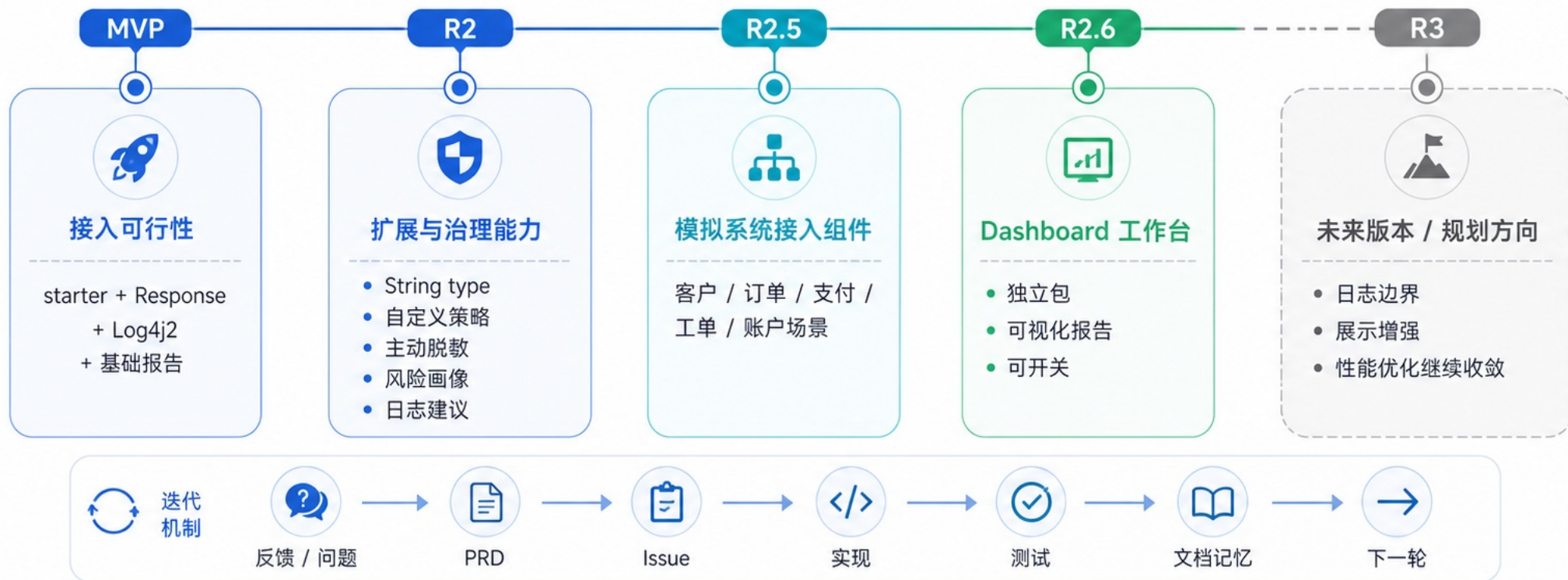
- AI 执行效率越高，越需要明确边界和验收。



SDD 解决做什么、不能做什么； **Issue** 拆成可执行任务； **TDD** 验证做得对不对。

版本迭代与需求收敛

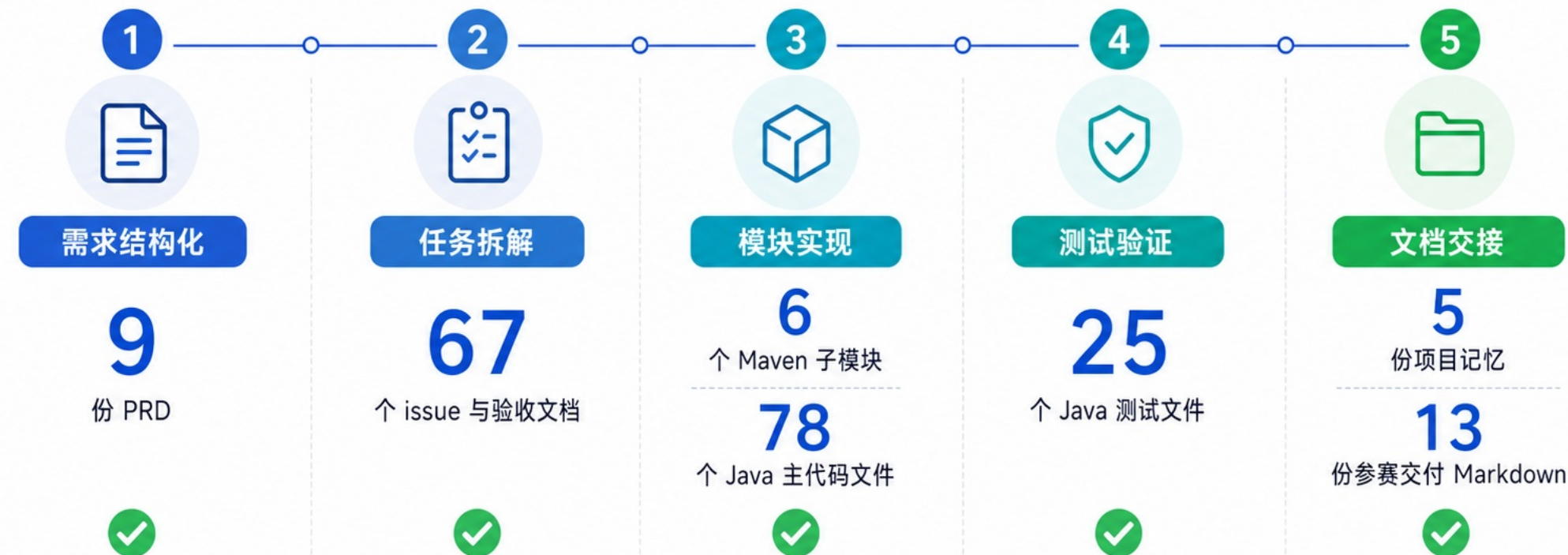
- Safe Output 的每一轮迭代，都围绕新的工程问题进行范围收敛和能力补齐。



版本迭代让 AI 产出从单次生成，转向**持续收敛**的工程过程。

交付完整性说明

纯 AI 项目的完整性，不只看一段可运行代码，更要看研发链路是否可复查、可验证、可交接。



不写不可复查的人天节省，而是用仓库产物证明 AI 覆盖了完整工程链路。

质量证明

纯AI项目必须实现单元测试、集成测试和黑盒验证的质量闭环。



关键结果



mvn test 通过



demo / starter / dashboard 指定模块测试通过



22/22 通过



20/20 通过



10/16 通过, 1 项失败, 5 项需人工介入验证



AI 将如实记录测试结果, 标记可自动完成和需人工二次确认的检查项。

Demo演示提供可视化认证



模拟接入系统



mock 业务域



接入 Dashboard



配置示范

1 协助快速接入

展示 YAML / 注解 / ignore、可配置参数、自定义策略。

接入配置

YAML 配置

注解配置

Ignore 配置

参数配置

自定义策略

YAML 配置示例

```
safe-output:
  enable: true
  domain: mock-order
  ignore-rules:
    - path: /actuator/health
      method: GET
    - path: /actuator/info
      params:
        sample-rate: 1.0
        max-log-length: 2000
  strategies:
    - name: mobile
      type: REGEX
      level: high
```

可配置参数

采样率 sample-rate 1.0

最大日志长度 max-log-length 2000

日志级别 log-level INFO

保存配置

自定义策略

策略名	类型	规则描述
mobile	REGEX	手机号脱敏
id-card	REGEX	身份证脱敏
bank-card	REGEX	银行卡脱敏

+ 新增策略

2 脱敏能力一致性

展示主动脱敏方法，业务系统接入后也可在代码内主动调用脱敏服务。

脱敏能力演示（主动脱敏）

调用示例（Java）

```
String raw = ["姓名"]张**
["手机"]138****5678 ["身份证"]
110*****2010";
String safe = maskService
.mask(raw);
// 业务代码内主动调用脱敏服务
```

脱敏结果预览

```
{
  "姓名": "张**",
  "手机": "138****5678",
  "身份证": "110*****2010"
}
```

脱敏策略生效情况

字段	策略	原始值（脱敏前）	结果（脱敏后）	命中规则
姓名	name-mask	张三	张**	✓
手机	mobile-mask	138****5678	138****5678	✓
身份证	id-card-mask	110*****2010	110*****2010	✓

3 强化治理演示

raw ignore 进入风险画像，Dashboard / 报告展示日志建议、接口风险、历史报告。

概览

概览

风险画像

日志建议

接口风险

历史报告

配置中心

治理 Dashboard

日志总数

12,842

高风险系数

342

建议系数

96

已处理建议

71

日志建议 Top 5

建议类型	示例摘要	风险等级	建议操作
敏感字段泄露	/api/order/create body 中存在未脱敏字段	高	查看建议
异常堆栈泄露	NullPointerException 堆栈包含类名	中	查看建议
接口响应包含明文	/api/user/info 响应包含明文字段	高	查看建议
日志级别不当	生产环境使用 DEBUG 日志	中	查看建议
Token 过期风险	Authentication 头包含未脱敏	高	查看建议

风险等级分布



最近历史报告

报告名称	生成时间	风险数	建议数	操作
20240526_治理报告.md	2024-05-26 10:32	36	18	下载
20240527_治理报告.md	2024-05-27 14:21	29	15	下载
20240526_治理报告.md	2024-05-26 09:11	41	22	下载



Demo 工作台: <http://122.51.95.83:8080/index.html>



Dashboard: <http://122.51.95.83:8080/safe-output/dashboard/index.html>



这条路线对应治理闭环：覆盖高频风险、发现遗漏线索、回补治理规则，最终形成报告。

能力边界

Safe Output 当前达到可演示、可集成验证的 MVP/R2 成熟度。



项目总结



边界清楚、过程可记录、结果可验证，纯 AI 项目也能形成有质量的工程交付。



AI 不替代工程纪律，而是加速工程纪律落到可交付作品中。